
Heart Disease Prediction

Machine Learning Project Summary
Presented By: Haitao Wang, Ritu Arora

Overview : Building a Robust Heart Disease Risk Prediction API

Goal: Develop, optimize, and deploy a stable ML model to classify patient risk of Heart Disease.

Final Model Champion: XGBoost Classifier (Selected for stability and performance).

Key Performance Metrics (V2):

- **ROC AUC:** 0.8403
- **Max F1 Score:** 0.3834

Critical Deployment Threshold: 0.1919

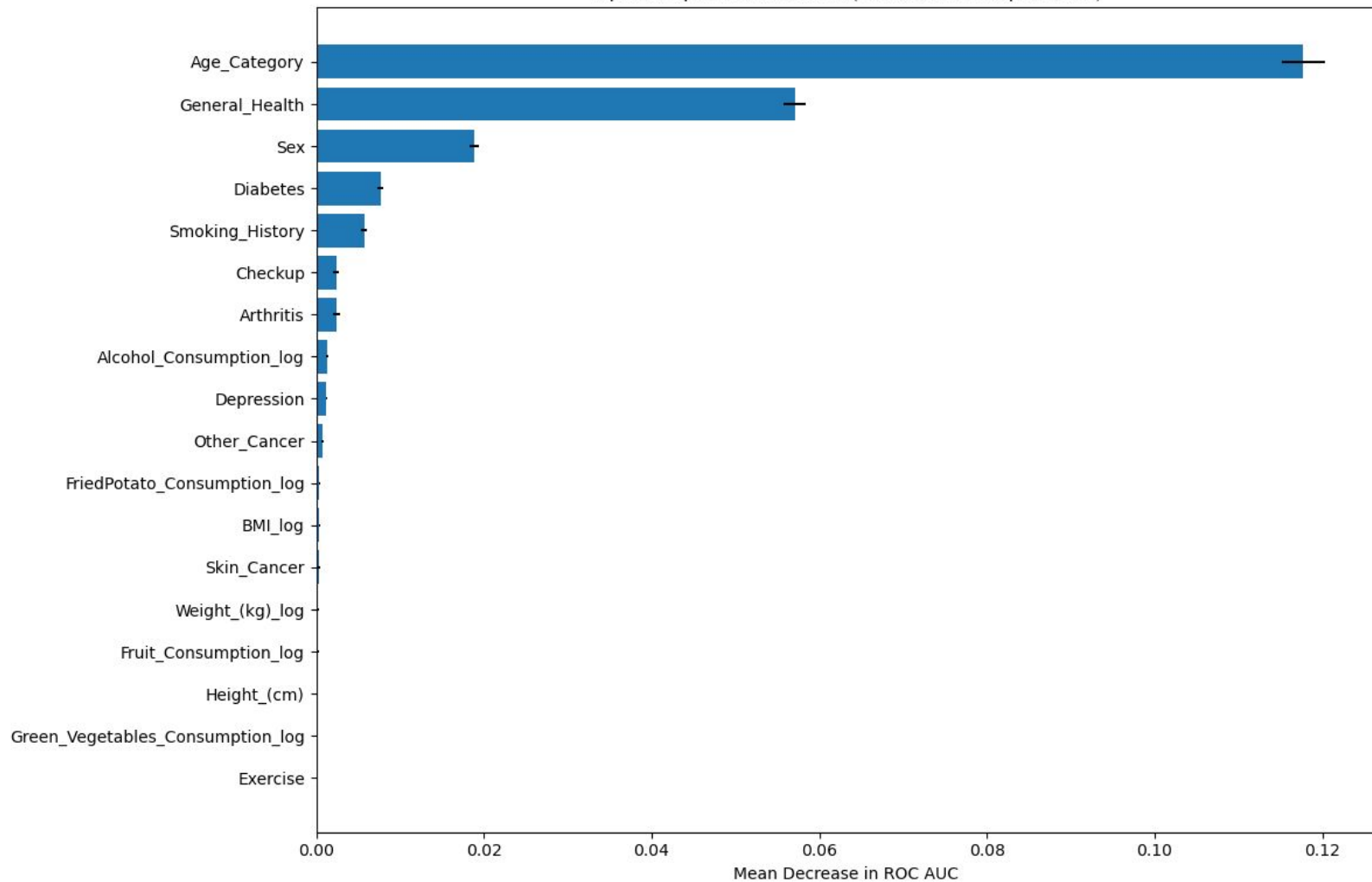
- *This is the probability threshold chosen to minimize false negatives (missed cases).*

The Data Science Challenge

The Core Problem: Unlocking Subtle Signal in Noisy Health Data

- **Challenge 1: High Imbalance:** Heart disease is a rare event in the general population, making true positive classification difficult.
- **Challenge 2: Redundant Features:** Initial models over-rely on categorical features (Age, Health) and ignore numerical data (BMI).
- **Goal:** Introduce **engineered features** to capture non-linear risk and enforce data fidelity from training to production.

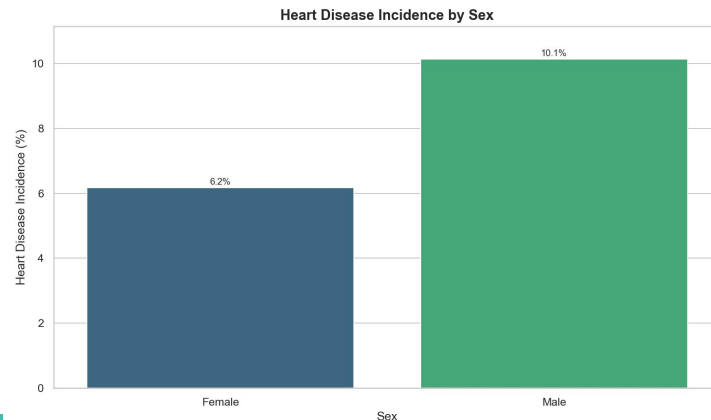
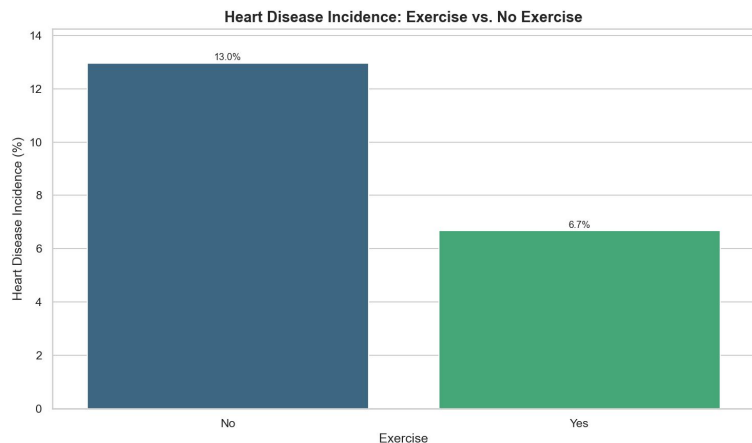
Top 20 Important Features (Permutation Importance)



EDA I: Baseline Risk Factors

Defining the Fundamental Risk Gradients

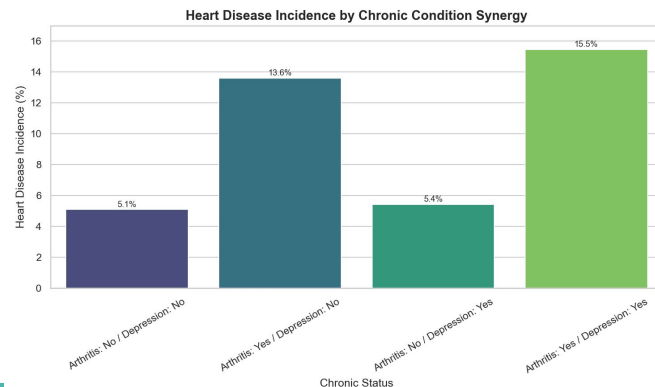
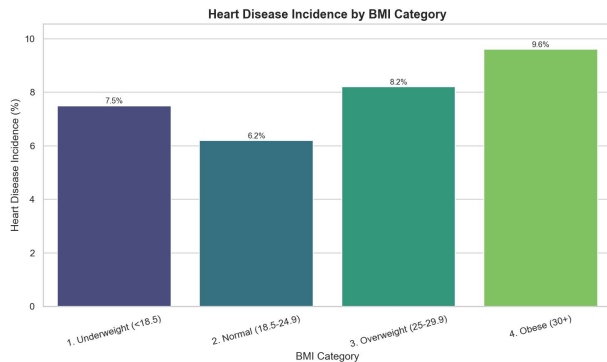
- **Lack of Exercise:** Incidence **doubles** from 6.7% (Yes) to 13.0% (No).
- **Sex:** Males (10.1%) show 60% higher incidence than females (6.2%).
- **Model Action:** These are foundational features that must be correctly handled as high-importance predictors.



EDA II: Complex Physiological Risk

Identifying Comorbidity Synergy

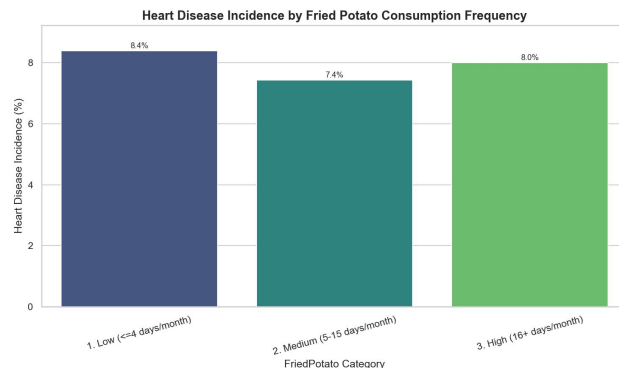
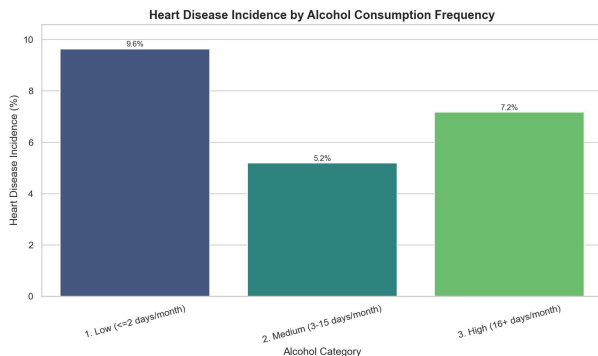
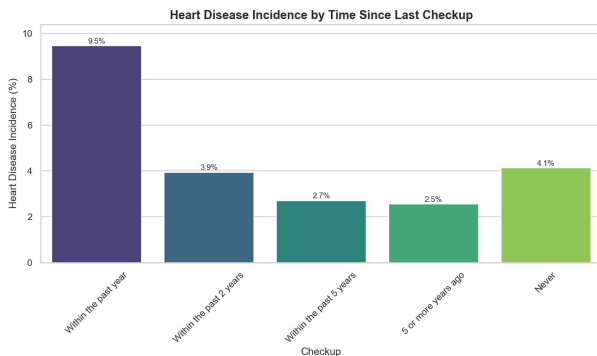
- **BMI:** Risk increases monotonically from Normal (6.2%) to Obese (9.6%). (The V2 intervention target.)
- **Arthritis Synergy:** Arthritis is the dominant risk factor. Incidence is 13.6% with Arthritis alone, rising to 15.5% when combined with Depression.
- **Model Action:** The BMI signal is strong but hidden; it requires an intervention. Comorbidity synergy is confirmed.



EDA III: Data Pitfalls and Noise

Data Traps: Reverse Causality & Convoluted Signals

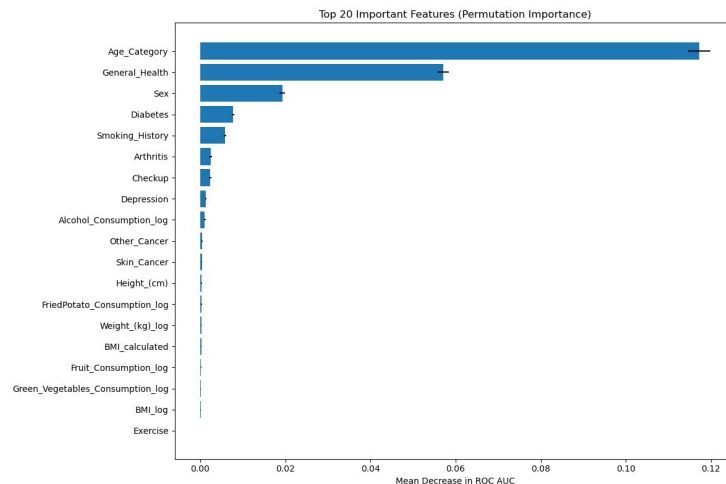
- **Reverse Causality:** Checkup "Within the past year" shows the highest incidence (9.5%). This signals existing illness, **not protection**.
- **Noisy Consumption:** Fried Potato and Alcohol features were flat or contradictory.
- **Model Action:** We must teach the model to treat "Checkup" as a risk factor and stabilize consumption features through **outlier capping**.



Diagnostic Failure: The Performance Plateau

The Model Couldn't See BMI

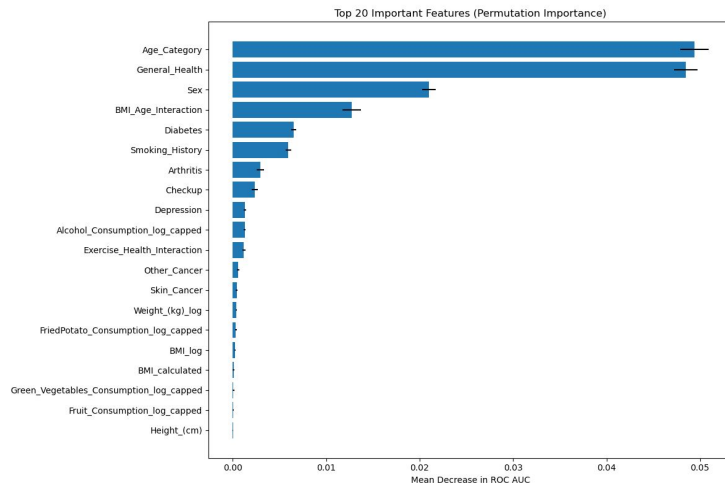
- **Initial Finding:** Permutation Importance showed BMI, Weight, and Height had **near-zero importance** (scores near 0.000).
- **The Plateau:** ROC AUC stalled at 0.8408.
- **The Cause:** The predictive signal from numerical health metrics was being completely overshadowed by the dominant categorical features (Age, Health, Sex).



The V2 Feature Strategy: Strategy

Designing Features to Unlock Hidden Risk

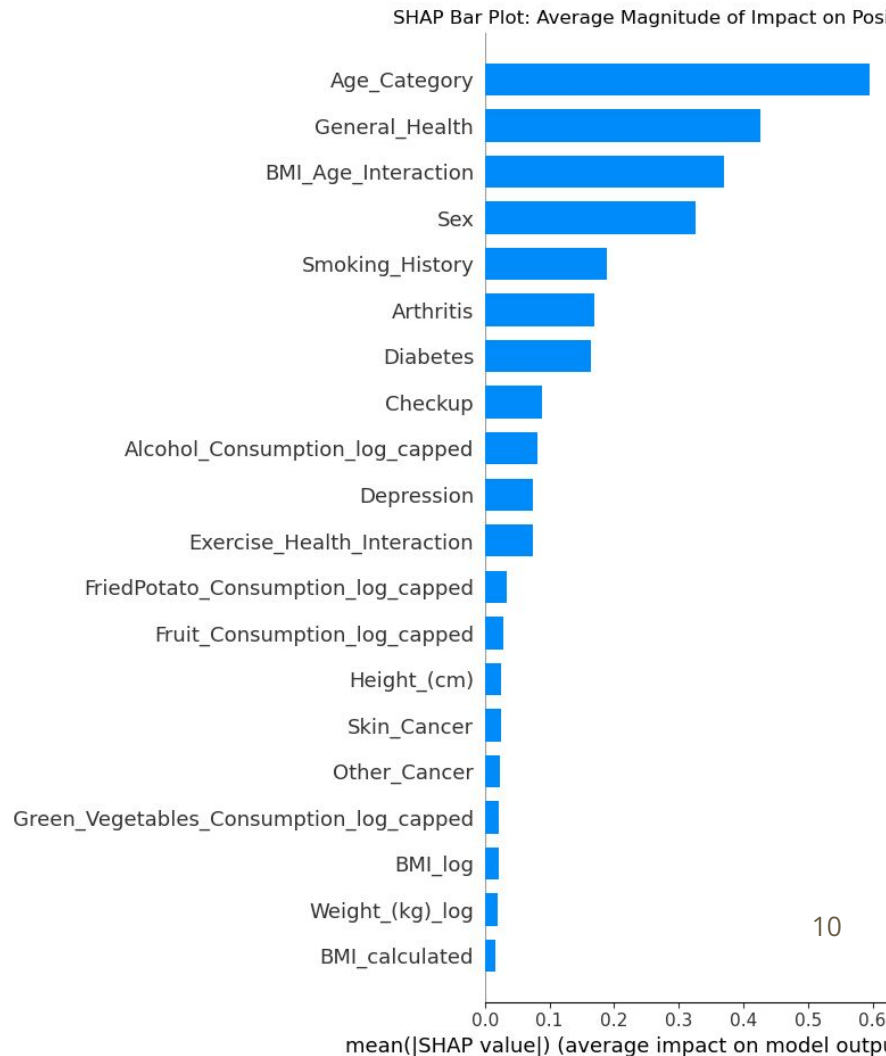
- **Strategy 1: Interaction Term**
 - **Goal:** Force the model to combine BMI and Age.
 - **Design:** BMI_Age_Interaction= $\log(\text{BMI}) \times \text{Age_Score}$
- **Strategy 2: Stabilization**
 - **Goal:** Make noisy consumption features useful
 - **Design:** Log-Transformation and Capping at 4.0 for all consumption features.
- **Strategy 3: Stability:** Switched training to the robust and stable XGBoost classifier due to severe computational issues with CatBoost.



The V2 Feature Strategy: Proof of Concept

Validating the BMI × Age Hypothesis

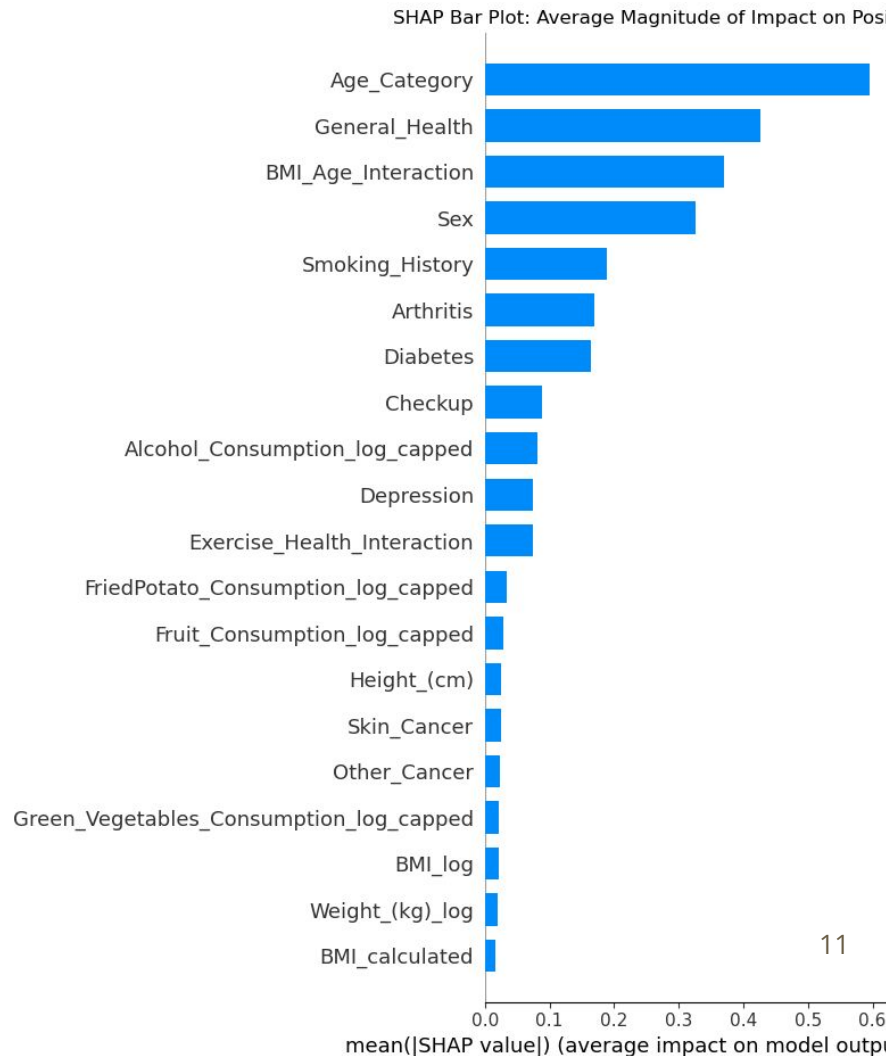
- **Mechanism:** This SHAP Dependence Plot shows the risk (SHAP value) contributed by the new interaction term.
- **Proof:** As the Age_Category increases (from blue/low to red/high), the impact of the BMI_Age_Interaction on risk **increases exponentially**.
- **Conclusion:** The engineered feature successfully forces the model to treat BMI as a high-risk factor only in older age groups.



V2 Feature Success: Permutation Impact

The Result: Feature Engineering Breaks the Plateau

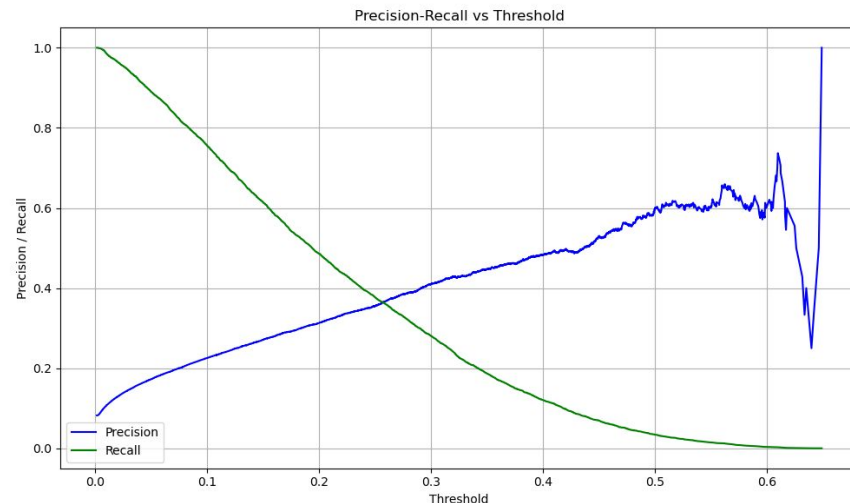
- **Key Success:** The **BMI_Age_Interaction** feature is immediately the **4th most important predictor**.
- **Impact:** It ranks higher than established medical risk factors like Diabetes and Smoking_History.
- **Stabilization Success:** Alcohol_Consumption_log_capped breaks into the Top 10, confirming outlier handling was effective.



Model Optimization: Why 0.1919?

Trading Precision for Patient Safety

- **Default Threshold (0.5):** Gave a strong Precision (0.5986) but terrible Recall (0.0341). 4905 True cases were missed.
- **Intervention Goal:** We prioritize **Recall** (catching cases that need intervention).
- **Solution:** We selected the point on the **Precision-Recall curve** that maximizes the F1-Score (the balance).



Model Optimization: Threshold Results

The Final Optimized Classification Matrix

- **Optimal Threshold:** 0.1919
- **F1 Score at Optimal Threshold:** 0.3819
- **Resulting Confusion Matrix:**
 - **True Positives (Cases Caught):** 2568
 - **False Negatives (Cases Missed):** 2510
- **Conclusion:** This is a strong, balanced result for a highly imbalanced, critical classification problem.

--- Model Performance Metrics ---

Default Threshold: 0.5

F1 score at Default Threshold: 0.0645

Confusion Matrix:

```
[[56561  116]
 [ 4905  173]]
```

Classification Report:

		precision	recall	f1-score	support
	0.0	0.9202	0.9980	0.9575	56677
	1.0	0.5986	0.0341	0.0645	5078

--- Optimal Threshold Metrics (Maximizing F1 Score) ---

Optimal Threshold: 0.1919

F1 Score at Optimal Threshold: 0.3819

Confusion Matrix at Optimal Threshold:

```
[[50876  5801]
 [ 2510  2568]]
```

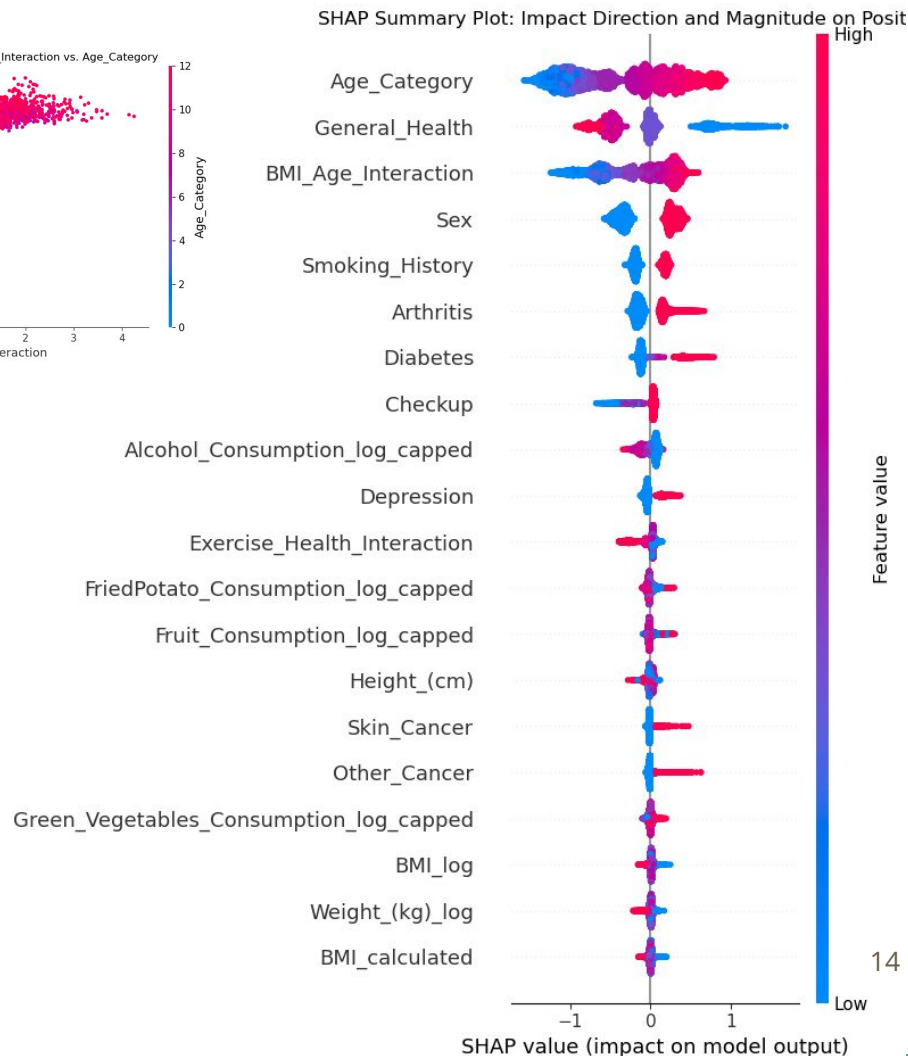
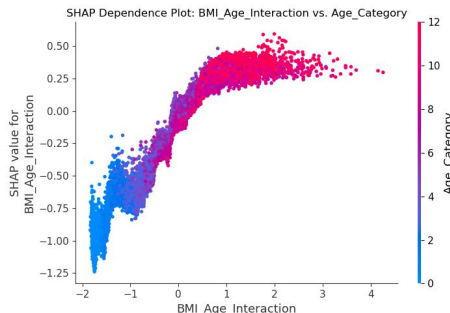
Classification Report at Optimal Threshold:

		precision	recall	f1-score	support
	0.0	0.9530	0.8976	0.9245	56677
	1.0	0.3068	0.5057	0.3819	5078

XAI: Model Trust and Interpretability

How the Final Model Makes Decisions (SHAP)

- **Directional Impact:** The model relies on Age (high values → high risk) and General_Health (low values → high risk) for its largest risk contributions.
- **Validation:** The new BMI_Age_Interaction pushes risk in the expected direction: **high values → high risk**.
- **Narrative:** We understand *how* the model works, which is essential for deployment trust.



The API Validation Crisis

The Final Hurdle: Feature Fidelity in Production

- **The Crisis:** After successful training, the API consistently predicted Low Risk for verified High Risk user inputs.
- **Root Cause: Ordinal Mapping Inversion.** The API's scoring code for features like General_Health did not match the zero-indexed scores learned by the saved scikit-learn preprocessor.
 - *Example: API scored 'Excellent' as 4 (High Risk), but the model expected 4 (Low Risk).*
- **Resolution:** We manually extracted and implemented the **verified ordinal mappings** from the saved artifact into app.py's scoring logic.

Final Proof: The Conservative Model

Testing the True Boundary

- **Test:** We used an extreme profile to find the 0.1919 threshold point.
- **Input Profile:** Age=80+, Diabetes=Yes, Sex=Male, Smoking=Yes, High BMI.
- **Result:** 25.32% probability (Safely above the threshold).
- **Key Insight:** The model is highly **conservative**. It requires the **maximal combination of severe, unambiguous risk factors** (Age80+ AND Diabetes Yes) to issue a High Risk signal.

Frontend Structure

```
frontend/
├── src/
│   ├── api/                # API logic
│   │   ├── client.js       # Configure HTTP client
│   │   └── endpoints.js    # / Define API endpoints
│   ├── assets/
│   ├── components/         # Reusable UI components
│   │   ├── Button.jsx
│   │   ├── Card.jsx
│   │   ├── ErrorBoundary.jsx # Error boundary
│   │   ├── Field.jsx       # Form field
│   │   ├── NavBar.jsx
│   │   ├── Segment.jsx     # Segment control
│   │   └── Toast.jsx       # Toast notification
│   ├── pages/              # Page-level components
│   │   ├── Home.jsx
│   │   ├── Calculator.jsx
│   │   ├── Results.jsx
│   │   ├── Resources.jsx
│   │   └── Disclaimer.jsx
│   ├── state/              # Global state management
│   │   └── ResultContext.jsx # Store & share results
│   ├── App.css             # Global CSS
│   ├── App.jsx             # Root component
│   ├── index.css           # Tailwind & CSS setup
│   ├── main.jsx            # Application entry
│   └── routes.jsx          # Routing config
├── index.html
├── package.json            # Project dependencies
├── postcss.config.cjs
├── tailwind.config.js
└── vite.config.js
```

pages/: route-level screens

components/: reusable UI building blocks

state/: global results storage

api/: backend communication

configs/: server/dependencies/styles

Data flow

1. Users fills form on pages/Calculator.jsx → clicks Calculate
2. Frontend validates & build payload → api/[clients.js](#) sends POST to backend [endpoints.js](#)
3. Backend runs ML → returns JSON
4. Frontend stores it in ResultContext → Results.jsx reads and renders → users can Download PDF

```
const schema = z.object({
  General_Health: z.enum(HEALTH_OPTIONS),
  Checkup: z.enum(CHECKUP_OPTIONS),
  Exercise: z.enum(["0", "1"]),
  Skin_Cancer: z.enum(["0", "1"]),
  Other_Cancer: z.enum(["0", "1"]),
  Depression: z.enum(["0", "1"]),
  Diabetes: z.enum(DIABETES_OPTIONS),
  Arthritis: z.enum(["0", "1"]),
  Sex: z.enum(SEX_OPTIONS),
  Age_Category: z.enum(AGE_GROUPS),

  Height_cm: z.coerce.number().min(0, { message: "Must be ≥ 0" }),
  Weight_kg: z.coerce.number().min(0, { message: "Must be ≥ 0" }),

  Smoking_History: z.enum(["0", "1"]),
  Alcohol_Consumption: z.coerce.number().min(0),
  Fruit_Consumption: z.coerce.number().min(0),
  Green_Vegetables_Consumption: z.coerce.number().min(0),
  FriedPotato_Consumption: z.coerce.number().min(0),
});
```

Calculator.jsx →
validates inputs

```

const onSubmit = async (v) => {
  const yn = (x) => (String(x) === "1" ? "Yes" : "No");

  const payload = {
    General_Health: v.General_Health,
    Checkup: v.Checkup,

    Exercise: yn(v.Exercise),
    Skin_Cancer: yn(v.Skin_Cancer),
    Other_Cancer: yn(v.Other_Cancer),
    Depression: yn(v.Depression),
    Arthritis: yn(v.Arthritis),
    Smoking_History: yn(v.Smoking_History),

    Diabetes: v.Diabetes,
    Sex: v.Sex,
    Age_Category: v.Age_Category,

    Height_cm: Number(v.Height_cm),
    Weight_kg: Number(v.Weight_kg),

    Alcohol_Consumption: Number(v.Alcohol_Consumption),
    Fruit_Consumption: Number(v.Fruit_Consumption),
    Green_Vegetables_Consumption: Number(v.Green_Vegetables_Consumption),
    FriedPotato_Consumption: Number(v.FriedPotato_Consumption),
  };

  const res = await postPredict(payload);

  setResult({ ...res, input: payload });
  nav("/results");
};

```

Calculator.jsx →
Builds payload

onSubmit(v)

client.js →
Sends POST to backend

```
export async function postPredict(payload) {  
  const { data } = await api.post("/predict", payload);  
  return normalizePredictResponse(data);  
}
```

```
function apiErrorMsg(err) {  
  if (err.response?.status === 422) return "Validation error: please check your inputs.";  
  if (err.code === "ECONNABORTED") return "Request timed out. Try again.";  
  if (err.response?.status >= 500) return "Server error. Please try again later.";  
  return "Network error. Please check your connection.";  
}
```

```
function normalizePredictResponse(raw) {  
  let probability = raw?.probability;  
  let prediction = raw?.prediction;  
  let threshold_used = raw?.threshold_used;  
  
  if (probability == null && raw?.risk_probability != null) {  
    probability = raw.risk_probability;  
  }  
  if (prediction == null && raw?.risk_level) {  
    prediction = raw.risk_level === "High" ? 1 : 0;  
  }  
  if (threshold_used == null) {  
    threshold_used = raw?.threshold ?? 0.5;  
  }  
  
  return {  
    probability: Number(probability ?? 0),  
    prediction: Number(prediction ?? 0),  
    threshold_used: Number(threshold_used ?? 0.5),  
  };  
}
```

```

export default function Results() {
  const { result } = useResult();
  const nav = useNavigate();

  if (!result) {
    return (
      <Card title="No result" className="min-h-[420px]">
        <p>Please run the calculator first.</p>
      </Card>
    );
  }
}

```

Results.jsx →
Renders and provides download

```

const handleDownload = () => {
  if (!input || Object.keys(input).length === 0) {
    alert("No form data found. Please re-run the assessment and try again.");
    return;
  }

  const doc = new jsPDF({ unit: "pt", format: "a4" });
  const lineHeight = 20;
  const pageWidth = doc.internal.pageSize.getWidth();
  const marginX = 48;
  let y = 56;

```

Why Log Assessments? Auditing & Monitoring

Capture every prediction for audit & monitoring

Store raw inputs + prediction results

Enables detection of model drift & debugging

Provides data for retraining & compliance

Dockerizing the Backend API

Containerized FastAPI app + PostgreSQL DB

Used `docker-compose` for orchestration

Enabled consistent, reproducible dev & deployment

Challenges:

- Database connection errors on startup
- Dependency injection with SQLAlchemy sessions

Challenges & Debugging

Bug: FastAPI attribute error on startup event decorator

Bug: Psycopg2 error saving numpy float types to DB

Debug tools:

- `docker-compose logs` for backend & DB containers
- PostgreSQL `\dt` and `SELECT` queries to verify data
- Type conversion & proper event handler usage

Architecture & Workflow

Thank you :)

Project Conclusion: CI/CD Pipeline

Production Integrity: Automation and Stability

- **CI (Continuous Integration):**
 - Implemented Pytest using the Low, High, and Threshold-Crossing profiles.
 - Tests must pass before deployment.
- **CD (Continuous Deployment):**
 - The GitHub Actions pipeline now autonomously builds and pushes separate **Frontend** and **Backend** Docker images to AWS ECR.
 - Deploys to AWS ECS, ensuring fast and reliable updates.
- **Final Status:** Project achieves high predictive power, interpretability, and production integrity.